



Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências – Bauru
Departamento de Computação

Sistema de Acompanhamento e Avaliação de Séries de TV

Modelagem conceitual, lógica e física de um banco de dados relacional

Banco de Dados I

Igor dos Reis Gomes – RA: 241025265

Nome do segundo integrante – RA: 000000000

Nome do terceiro integrante – RA: 000000000

Nome do quarto integrante – RA: 000000000

Bauru – SP
2026

Sumário

Como utilizar este documento	1
1 Modelagem Conceitual	2
1.1 Descrição do minimundo	2
1.2 Regras de negócio	3
1.3 Entidades e atributos	4
1.3.1 usuario	5
1.3.2 serie	5
1.3.3 temporada (entidade fraca)	5
1.3.4 episodio (entidade fraca)	5
1.3.5 genero	6
1.3.6 pessoa	6
1.3.7 avaliacao (entidade genérica)	6
1.3.8 avaliacao_serie, avaliacao_temporada, avaliacao_episodio	6
1.3.9 comentario	6
1.4 Relacionamentos e cardinalidades	7
1.5 Generalização e especialização	7
1.6 Agregação	8
1.7 Atributos derivados	8
1.8 Diagrama entidade-relacionamento	8
2 Modelagem Lógica	16
2.1 Regras de mapeamento aplicadas	16
2.1.1 Tratamento da especialização	18
2.1.2 Tratamento das entidades fracas	20
2.2 Esquema relacional	20
2.3 Dependências funcionais	22
2.4 Normalização	24
2.4.1 Primeira forma normal	25
2.4.2 Segunda forma normal	25
2.4.3 Terceira forma normal	26
2.4.4 Verificação do esquema final	26
2.5 Diagrama lógico	27
3 Modelagem Física	31
3.1 Convenções de nomenclatura e criação do esquema	31
3.2 Definição de dados (DDL)	31
3.3 Restrições de integridade	31
3.4 Gatilhos	32
3.5 Visões	33
3.6 Carga de dados (DML)	34

3.7	Consultas de exemplo	34
3.7.1	Consulta 1 – Ranking de séries por nota média	34
3.7.2	Consulta 2 – Atividade recente de quem o usuário segue	34
3.7.3	Consulta 3 – Séries na lista de interesse ainda não iniciadas	35
3.8	Rastreabilidade das regras de negócio	35

A Divisão de trabalho e convenções do grupo 36

A.1	Atribuições	36
A.2	Contrato de nomes	36
A.3	Verificação antes da entrega	36

Como utilizar este documento

Este arquivo já contém a **modelagem conceitual completa** (Capítulo 1). Os Capítulos 2 e 3 estão estruturados, mas precisam ser escritos pelos demais integrantes.

O Capítulo 1 não é apenas a primeira etapa do trabalho: ele **fixa o vocabulário** do projeto. Os nomes de entidades, atributos, tabelas e colunas definidos ali valem para todo o restante do documento e para o script SQL. Se alguma etapa posterior precisar alterar um nome, a alteração deve ser feita *também* no Capítulo 1, para que o documento não se contradiga – um esquema físico que não corresponde ao diagrama conceitual é o erro mais comum e mais caro deste tipo de trabalho.

Os trechos marcados como *Orientação* explicam o que se espera em cada seção. Eles podem ser ocultados alterando-se, no início do arquivo, `\orientacoestrue` para `\orientacoesfalse`. O Apêndice de divisão de trabalho também desaparece nesse caso, pois é material interno do grupo.

Capítulo 1

Modelagem Conceitual

Este capítulo descreve o minimundo escolhido pelo grupo, enuncia as regras de negócio que o banco de dados deve sustentar e apresenta o diagrama entidade-relacionamento (DER) correspondente.

O domínio foi escolhido por concentrar, em um cenário familiar a qualquer usuário, as construções que a modelagem conceitual precisa exercitar: uma hierarquia de composição em três níveis, relacionamentos muitos-para-muitos com atributos próprios, um grafo social que relaciona um conjunto de entidades a ele mesmo e um ato, o de avaliar, que incide indistintamente sobre três tipos de alvo. Todas elas decorrem da forma como as aplicações de acompanhamento de séries funcionam.

1.1 Descrição do minimundo

O sistema modelado é uma rede social voltada ao acompanhamento e à avaliação de séries de televisão. Ele não exibe conteúdo audiovisual: registra e organiza o que as pessoas assistem e o que elas acham do que assistiram. O paralelo mais próximo são as plataformas de catalogação de filmes e de livros, como Letterboxd e Goodreads, transposto para o formato seriado, no qual a obra se subdivide em unidades menores e cada uma delas pode ser assistida e avaliada separadamente.

O acervo do sistema é composto por **séries**, cada uma organizada em **temporadas**, que por sua vez se dividem em **episódios**. Quase toda interação do usuário acontece em um desses três níveis, e é por isso que a hierarquia organiza o restante do modelo. Cada série é classificada em um ou mais **gêneros** e conta com a participação de **pessoas** do meio audiovisual, que podem atuar como atores, diretores, roteiristas ou criadores. Uma mesma pessoa pode exercer funções diferentes em séries diferentes, e até na mesma série.

Os **usuários** do sistema mantêm uma relação individual com cada série do acervo. Essa relação tem duas faces complementares. A primeira é a **intenção declarada**: o usuário marca a série como algo que pretende assistir, que está assistindo ou que abandonou. A segunda é o **histórico efetivo**: o usuário registra, episódio a episódio, o que de fato assistiu e em que data. As duas faces não são redundantes. Uma série pode estar marcada como “pretendo assistir” sem nenhum episódio registrado, e o progresso real de quem está assistindo só pode ser calculado a partir dos registros individuais.

Sobre qualquer um dos três níveis da hierarquia, o usuário pode publicar uma **avaliação**: uma nota, um texto crítico, ou ambos. Avaliar uma série inteira, uma temporada específica ou um episódio isolado são atos de natureza idêntica, em que mudam apenas o alvo e o grau de detalhe, e por isso o modelo os trata como um único conceito especializado em três formas.

A camada social se organiza em torno dessas avaliações. Um usuário pode **seguir** outros usuários, o que constitui seu acompanhamento de atividade; pode **curtir** avaliações alheias; e

pode **comentar** nelas, inclusive respondendo a comentários de outras pessoas. A curtida e o comentário incidem sobre o *ato de avaliar*, isto é, sobre a associação entre um usuário e um item do acervo.

O minimundo organiza-se, portanto, em três pilares: o **acervo** (o dado sobre a obra), o **acompanhamento** (a relação entre usuário e obra) e o **social** (a relação entre usuários, mediada por avaliações).

1.2 Regras de negócio

***Orientação:** Cada regra abaixo precisa ser rastreável até uma construção concreta do banco. Ao escrever os Capítulos 2 e 3, cite o código da regra (RN01, RN02, ...) ao lado da restrição, do gatilho ou da chave que a implementa. A Seção 3.8 traz a tabela de rastreabilidade a ser preenchida.*

Usuários e relação social

- RN01** Cada usuário é identificado por um **username** único no sistema, composto apenas por letras minúsculas, dígitos e sublinhado, com 3 a 30 caracteres. O endereço de e-mail também é único.
- RN02** Um usuário pode seguir outros usuários. A relação é **unidirecional**: seguir alguém não implica ser seguido de volta.
- RN03** Nenhum usuário pode seguir a si mesmo.
- RN04** A exclusão de um usuário implica a exclusão de todo o seu histórico: estados declarados, registros de exibição, avaliações, curtidas e comentários.

Acervo

- RN05** Toda série pertence a pelo menos um gênero e pode pertencer a vários.
- RN06** Toda temporada pertence a exatamente uma série, e seu número é único *dentro* daquela série. Não existe temporada fora de uma série.
- RN07** Todo episódio pertence a exatamente uma temporada, e seu número é único *dentro* daquela temporada. Não existe episódio fora de uma temporada.
- RN08** A exclusão de uma série implica a exclusão de suas temporadas e, por consequência, de seus episódios.
- RN09** Uma pessoa pode participar de várias séries, e em cada série pode exercer mais de uma função (ator, diretor, roteirista ou criador).
- RN10** O nome do personagem só faz sentido para a função de ator; nas demais funções, esse dado é inexistente.

Acompanhamento

- RN11** Um usuário pode registrar o mesmo episódio mais de uma vez, em datas diferentes. Cada registro é um evento independente, o que permite representar reexibições.
- RN12** A data de um registro de exibição não pode ser futura.

- RN13** Um usuário declara, para cada série, no máximo **um** estado, entre: *watchlist* (pretende assistir), *assistindo* ou *abandonada*. Séries sobre as quais o usuário nada declarou simplesmente não possuem estado registrado.
- RN14** Registrar a exibição de um episódio de uma série implica que o usuário passa a estar *assistindo* aquela série. Se ele havia declarado *watchlist*, o estado é promovido automaticamente.

Avaliações

- RN15** Uma avaliação é publicada por exatamente um usuário e incide sobre exatamente um alvo, que é **ou** uma série, **ou** uma temporada, **ou** um episódio. Não existe avaliação sem alvo, nem avaliação com mais de um alvo.
- RN16** A nota atribuída varia de 0,5 a 5,0, em incrementos de meio ponto. A nota é opcional.
- RN17** Uma avaliação não pode ser inteiramente vazia: deve conter ao menos uma nota ou ao menos um texto.
- RN18** Um usuário só pode avaliar um episódio que ele já tenha registrado como assistido.
- RN19** Um usuário possui no máximo uma avaliação por alvo. Reavaliar significa atualizar a avaliação existente, não criar outra.
- RN20** O autor pode sinalizar que sua avaliação contém spoiler, para que a aplicação oculte o texto por padrão.

Interação sobre avaliações

- RN21** Um usuário pode curtir uma avaliação uma única vez.
- RN22** Nenhum usuário pode curtir a própria avaliação.
- RN23** Uma avaliação pode receber comentários de qualquer usuário. Um comentário pode ser resposta a outro comentário, desde que ambos pertençam à mesma avaliação.
- RN24** A exclusão de uma avaliação implica a exclusão de suas curtidas e de seus comentários.

1.3 Entidades e atributos

O modelo possui **onze conjuntos de entidades**. Os relacionamentos do tipo muitos-para-muitos, que darão origem a tabelas próprias apenas na etapa lógica, estão descritos na Seção 1.4 e não são contados aqui.

Orientação: A notação de chave usada nas tabelas a seguir: **PK** identificador primário; **PP** chave parcial (entidade fraca, única apenas dentro do proprietário); **U** valor único no conjunto; **D** discriminador de especialização. Atributos marcados como obrigatórios não admitem valor nulo.

1.3.1 usuario

Representa a pessoa que utiliza o sistema.

Atributo	Tipo	Chave	Obrig.	Descrição
id_usuario	inteiro	PK	sim	Identificador interno
username	texto(30)	U	sim	Identificador público; minúsculas, dígitos e _
email	texto(120)	U	sim	Endereço de contato
nome_exibicao	texto(60)		não	Nome apresentado na interface
bio	texto		não	Descrição livre escrita pelo usuário
data_cadastro	data		sim	Data de entrada no sistema

1.3.2 serie

Representa a obra audiovisual seriada.

Atributo	Tipo	Chave	Obrig.	Descrição
id_serie	inteiro	PK	sim	Identificador interno
titulo	texto(150)		sim	Título de exibição
titulo_original	texto(150)		não	Título no idioma de origem
sinopse	texto		não	Resumo da obra
ano_estreia	inteiro		não	Ano da primeira exibição
situacao	texto(15)		sim	em_exibicao, encerrada ou cancelada
pais_origem	texto(2)		não	Código de país de duas letras

1.3.3 temporada (entidade fraca)

Agrupar episódios dentro de uma série. É uma **entidade fraca**: seu número não identifica nada sozinho, já que “temporada 2” só faz sentido quando se sabe de qual série se trata. A identificação depende, portanto, do relacionamento identificador com **serie**.

Atributo	Tipo	Chave	Obrig.	Descrição
numero	inteiro	PP	sim	Número da temporada dentro da série; maior que zero
titulo	texto(100)		não	Nome da temporada, quando houver
data_estreia	data		não	Data de estreia da temporada

1.3.4 episodio (entidade fraca)

Unidade mínima de conteúdo e alvo mais frequente de registro e avaliação. Também é entidade fraca, dependente de **temporada** pelos mesmos motivos.

Atributo	Tipo	Chave	Obrig.	Descrição
numero	inteiro	PP	sim	Número do episódio dentro da temporada; maior que zero
titulo	texto(150)		sim	Título do episódio
duracao_min	inteiro		não	Duração em minutos; maior que zero
data_exibicao	data		não	Data da exibição original
sinopse	texto		não	Resumo do episódio

1.3.5 genero

Atributo	Tipo	Chave	Obrig.	Descrição
id_genero	inteiro	PK	sim	Identificador interno
nome	texto(40)	U	sim	Nome do gênero (Drama, Comédia, ...)

1.3.6 pessoa

Profissional do meio audiovisual envolvido na produção de séries.

Atributo	Tipo	Chave	Obrig.	Descrição
id_pessoa	inteiro	PK	sim	Identificador interno
nome	texto(120)		sim	Nome da pessoa
data_nascimento	data		não	Data de nascimento
nacionalidade	texto(60)		não	Nacionalidade declarada

1.3.7 avaliacao (entidade genérica)

Representa o ato de avaliar: a manifestação de um usuário sobre um item do acervo. É a **superclasse** de uma especialização detalhada na Seção 1.5.

Atributo	Tipo	Chave	Obrig.	Descrição
id_avaliacao	inteiro	PK	sim	Identificador interno
nota	decimal(2,1)		não	De 0,5 a 5,0, em múltiplos de 0,5 (RN16)
texto	texto		não	Resenha escrita pelo usuário
contem_spoiler	booleano		sim	Sinaliza conteúdo que revela enredo (RN20)
data_publicacao	data/hora		sim	Momento da publicação
tipo_alvo	texto(10)	D	sim	serie, temporada ou episodio
total_curtidas	inteiro		sim	Atributo derivado; ver Seção 1.7

1.3.8 avaliacao_serie, avaliacao_temporada, avaliacao_episodio

As três subclasses da especialização. Nenhuma delas possui atributos próprios além da ligação com o alvo correspondente: o que as distingue é *sobre o que* incidem. Cada instância de **avaliacao** corresponde a exatamente uma instância de uma das três.

1.3.9 comentario

Manifestação textual de um usuário a respeito de uma avaliação.

Atributo	Tipo	Chave	Obrig.	Descrição
id_comentario	inteiro	PK	sim	Identificador interno
texto	texto		sim	Conteúdo do comentário; não pode ser vazio
data_publicacao	data/hora		sim	Momento da publicação

1.4 Relacionamentos e cardinalidades

A Tabela 1.1 enumera os relacionamentos do modelo na notação (*mín*, *máx*), isto é, a participação mínima e máxima de cada entidade no relacionamento. Participação mínima igual a 1 indica **participação total**.

Tabela 1.1: Relacionamentos do modelo conceitual

Relacionamento	Entidades	Cardinalidade	Atributos próprios
segue	usuario / usuario	(0,N) : (0,N)	data_inicio
possui_temporada	serie / temporada	(1,N) : (1,1)	nenhum
possui_episodio	temporada / episodio	(1,N) : (1,1)	nenhum
classifica	serie / genero	(1,N) : (0,N)	nenhum
participa	persona / serie	(0,N) : (0,N)	funcao, personagem
declara_estado	usuario / serie	(0,N) : (0,N)	estado, data_atualizacao
registra	usuario / episodio	(0,N) : (0,N)	data_assistido
escreve	usuario / avaliacao	(0,N) : (1,1)	nenhum
curte	usuario / avaliacao	(0,N) : (0,N)	data_curtida
recebe	avaliacao / comentario	(0,N) : (1,1)	nenhum
comenta	usuario / comentario	(0,N) : (1,1)	nenhum
responde	comentario / comentario	(0,1) : (0,N)	nenhum
incide_serie	avaliacao_serie / serie	(1,1) : (0,N)	nenhum
incide_temporada	avaliacao_temporada / temporada	(1,1) : (0,N)	nenhum
incide_episodio	avaliacao_episodio / episodio	(1,1) : (0,N)	nenhum

Os três relacionamentos *incide_** ligam cada subclasse da especialização ao item do acervo que ela avalia. Eles decorrem da Seção 1.5: se a avaliação incide sobre exatamente um alvo (RN15), cada subclasse precisa de uma ligação com esse alvo. A participação (1,1) do lado da subclasse é o que garante que nenhuma avaliação fique sem alvo.

O *segue* é um **auto-relacionamento**: as duas pontas são o mesmo conjunto de entidades, e por isso exigem **papéis** distintos, *seguidor* e *seguido*. Sem os papéis, o diagrama seria ambíguo, pois não haveria como saber qual das pontas representa quem segue.

O *registra* não possui restrição de unicidade por par usuário-episódio, o que é incomum em relacionamentos desse tipo. A escolha implementa a RN11: cada registro representa um evento de exibição datado, de modo que registrar o mesmo episódio duas vezes descreve uma reexibição. O relacionamento passa a funcionar como um histórico.

O *responde* é um segundo auto-relacionamento, desta vez de grau 1:N: um comentário tem no máximo um comentário-pai e pode ter vários comentários-filho. A participação mínima zero em ambas as pontas reflete que comentários de primeiro nível não têm pai, e comentários sem resposta não têm filhos.

1.5 Generalização e especialização

A entidade *avaliacao* é especializada em *avaliacao_serie*, *avaliacao_temporada* e *avaliacao_episodio*. A especialização é **total** e **disjunta**:

- **Total**, porque toda avaliação incide obrigatoriamente sobre algum alvo (RN15). Não existe instância de *avaliacao* que não pertença a uma das três subclasses.
- **Disjunta**, porque os alvos são mutuamente exclusivos: uma mesma avaliação não pode incidir simultaneamente sobre uma série e sobre um episódio. Avaliar a série e avaliar um de seus episódios são dois atos distintos, que geram duas avaliações.

O atributo `tipo_alvo` atua como **discriminador**, indicando a qual subclasse cada instância pertence.

***Orientação:** Esta é a decisão de projeto mais importante do capítulo, e deve ser retomada na Seção 2.1 quando o mapeamento for feito. A alternativa descartada é descrita abaixo – vale reproduzir esse raciocínio na apresentação, porque demonstra que a escolha foi consciente.*

A alternativa considerada foi tratar `avaliacao` como entidade única, com três atributos de ligação opcionais, dos quais exatamente um estaria preenchido. Essa solução é comum em sistemas reais, porque simplifica as consultas: todas as avaliações ficam em uma só tabela. Ela foi descartada aqui por duas razões. A primeira é semântica: uma entidade cujos atributos de ligação são quase todos nulos descreve mais a acomodação de implementação escolhida do que o minimundo. A segunda é de integridade: garantir que exatamente um dos três atributos esteja preenchido exige uma restrição composta que o modelo conceitual não consegue enunciar, apenas comentar. Com a especialização, a exclusividade passa a ser uma propriedade *estrutural* do modelo.

1.6 Agregação

A entidade `avaliacao` é, conceitualmente, o resultado de uma associação entre um usuário e um item do acervo, ou seja, um **relacionamento reificado**. O modelo a trata como uma **agregação**, e isso tem uma consequência concreta: os relacionamentos `curte` e `recebe` ligam usuários à *associação inteira*.

A distinção é observável no comportamento do sistema. As alternativas seriam ligar a curtida à série, um relacionamento entre `usuario` e `serie`, ou ao autor, um relacionamento entre `usuario` e `usuario`. O que o usuário `curte` é o fato de *aquela pessoa específica ter avaliado aquele item específico daquele modo*. Se a avaliação for excluída, a curtida perde o objeto e deixa de existir (RN24); presa à série ou ao autor, ela sobreviveria à exclusão.

No diagrama, portanto, `curte` e `recebe` partem do retângulo que envolve a agregação, e não de uma das entidades participantes.

1.7 Atributos derivados

O atributo `total_curtidas` de `avaliacao` é **derivado**: seu valor é obtido pela contagem das instâncias de `curte` associadas àquela avaliação. Atributos derivados costumam ser omitidos do modelo, justamente por serem redundantes.

Optou-se por mantê-lo por uma razão prática: a ordenação de avaliações por popularidade é uma das operações mais frequentes do sistema, e recalculá-lo a cada leitura é caro. A redundância é aceita conscientemente, mas exige contrapartida: um mecanismo que garanta que o valor armazenado jamais divirja da contagem real. A Seção 3.4 especifica esse mecanismo na forma de um gatilho.

1.8 Diagrama entidade-relacionamento

***Orientação:** O diagrama deve ser produzido no **brModelo**, em notação de Chen, exportado como imagem e salvo em `images/der-conceitual.png`. Depois, basta descomentar o bloco de figura abaixo. O uso de uma ferramenta CASE atende ao item 7 do conteúdo programático da disciplina. Confiram, ao desenhar: (i) entidades fracas com retângulo*

duplo e relacionamento identificador com losango duplo; (ii) papéis nomeados nos dois auto-relacionamentos; (iii) triângulo de especialização com as marcações de totalidade e disjunção; (iv) retângulo tracejado envolvendo a agregação; (v) atributo derivado com elipse tracejada.

A Figura 1.1 apresenta o diagrama completo. Como a densidade de elementos obriga a uma redução considerável na página, as Figuras 1.2 a 1.6 ampliam, uma a uma, as quatro regiões que concentram as construções mais delicadas do modelo. Entidades que aparecem em mais de uma figura são as fronteiras entre regiões, repetidas de propósito para que cada ampliação se leia sozinha.

A especificação textual das Seções 1.3 a 1.7 é suficiente e não ambígua para a construção do diagrama: toda entidade, todo atributo com seu tipo e caráter de chave, toda cardinalidade e toda construção especial (entidade fraca, especialização, agregação, atributo derivado) está enunciada. Diagrama e texto devem permanecer coerentes: qualquer renomeação feita adiante precisa aparecer nos dois.

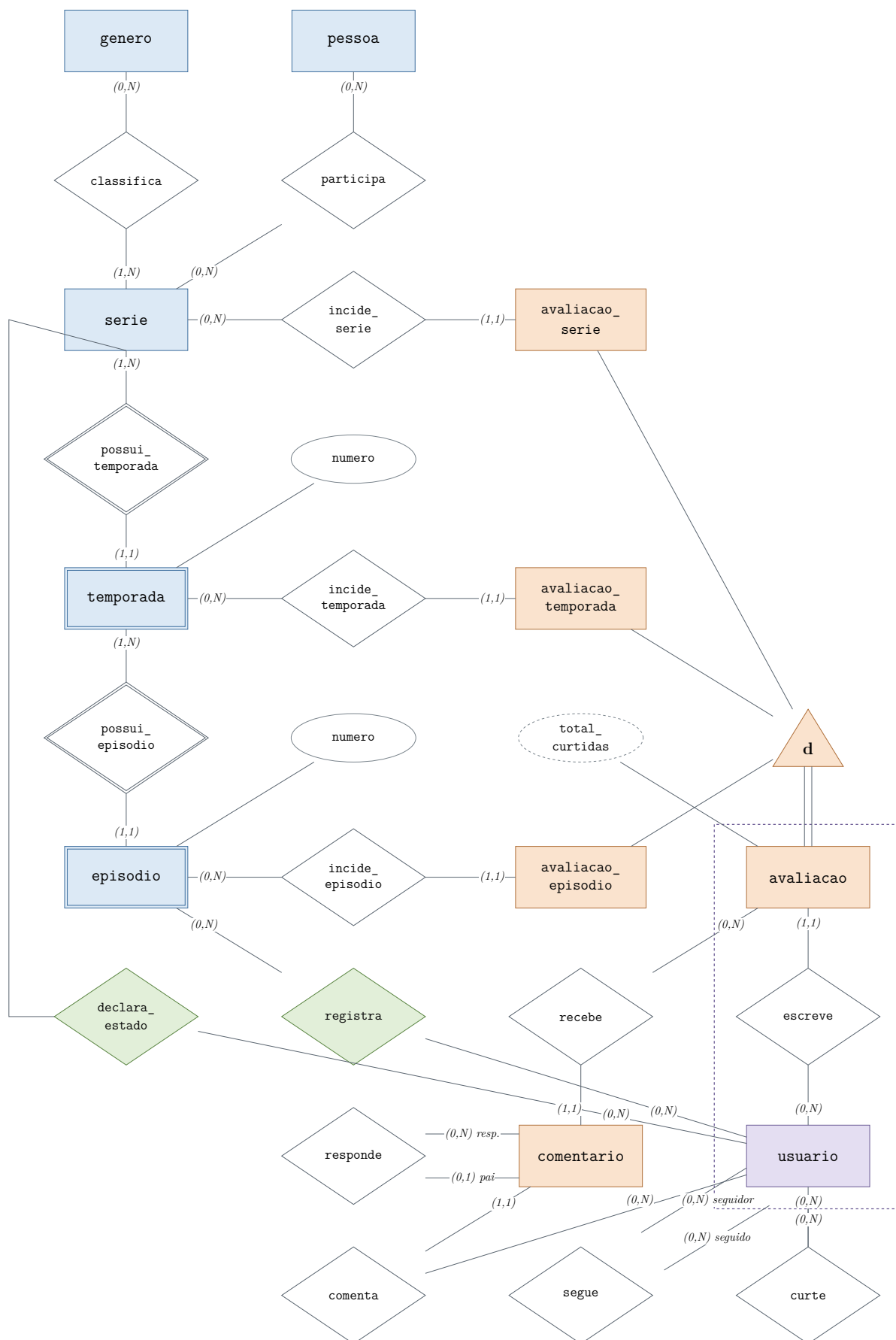


Figura 1.1: Visão estrutural do diagrama entidade-relacionamento. Mostra as onze entidades, os quinze relacionamentos com suas cardinalidades e as construções especiais do modelo. Os atributos comuns foram omitidos por legibilidade e estão especificados, com tipo e caráter de chave, nas tabelas da Seção 1.3; permanecem no diagrama apenas os que carregam notação própria, a chave parcial das duas entidades **frase** e **usuario** e o atributo derivado **total_curtidas**.

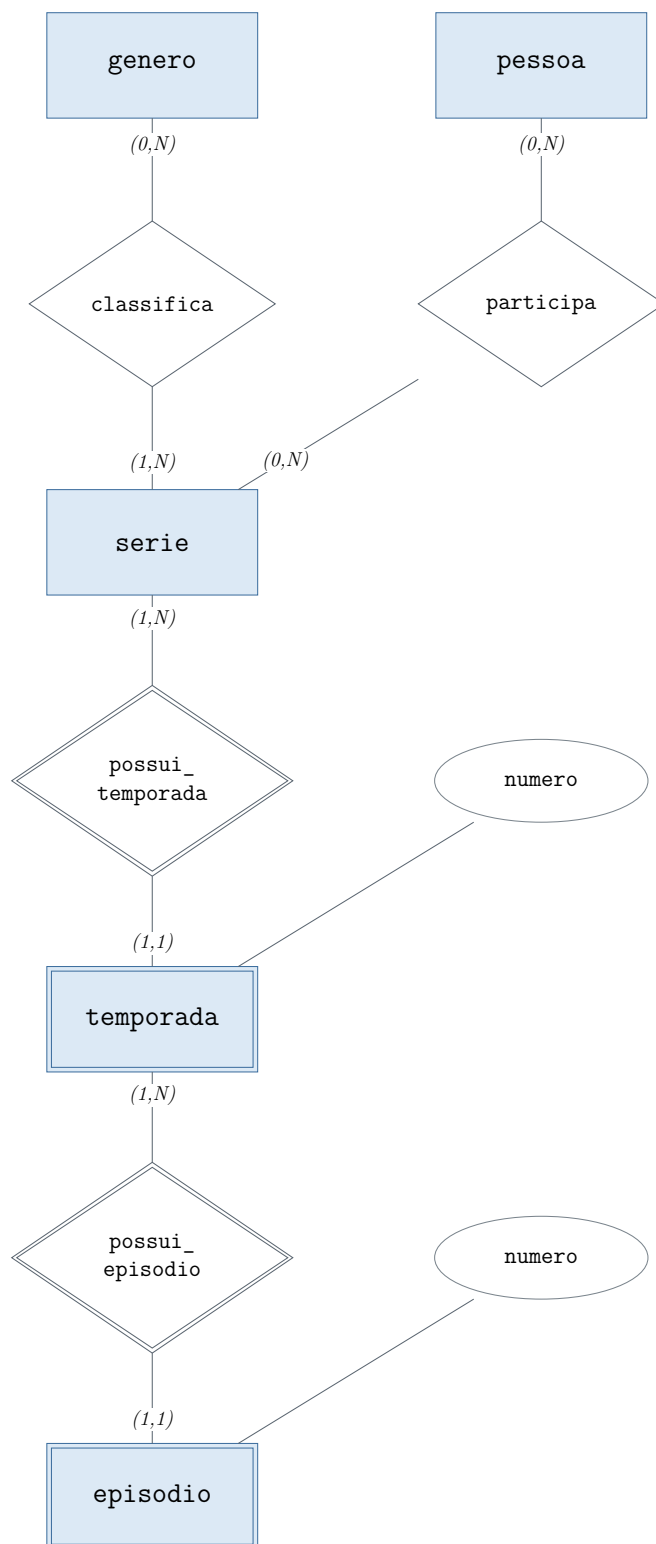


Figura 1.2: Ampliação da região do acervo. A cadeia série–temporada–episódio, com as duas entidades fracas em retângulo duplo, seus relacionamentos identificadores em losango duplo e as chaves parciais **numero**.

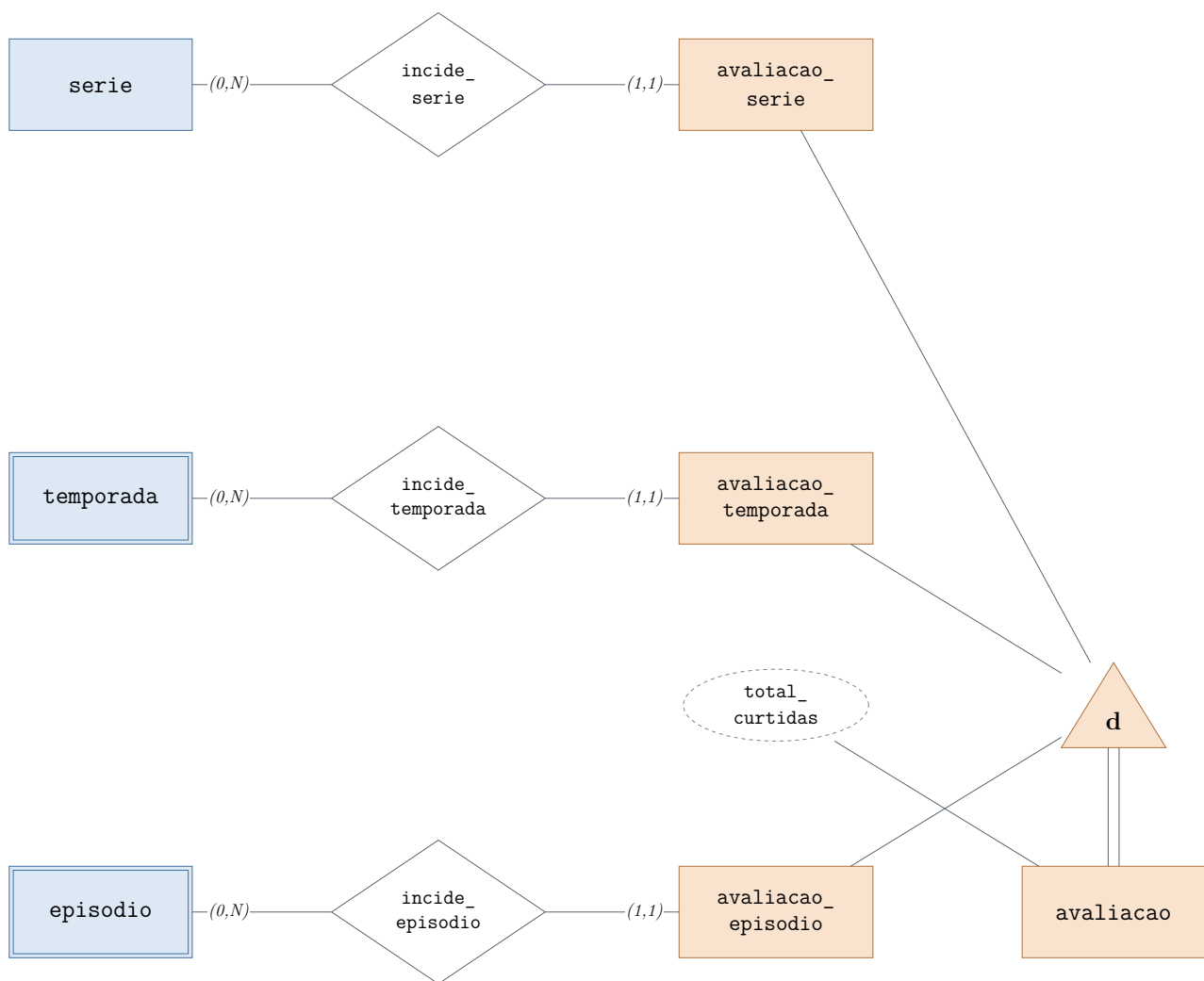


Figura 1.3: Ampliação da especialização. Cada subclasse liga-se ao seu alvo por um relacionamento **incide_*** com participação (1,1), e o triângulo marcado com **d** recebe de **avaliacao** uma linha dupla: disjunta pela marca, total pela linha.

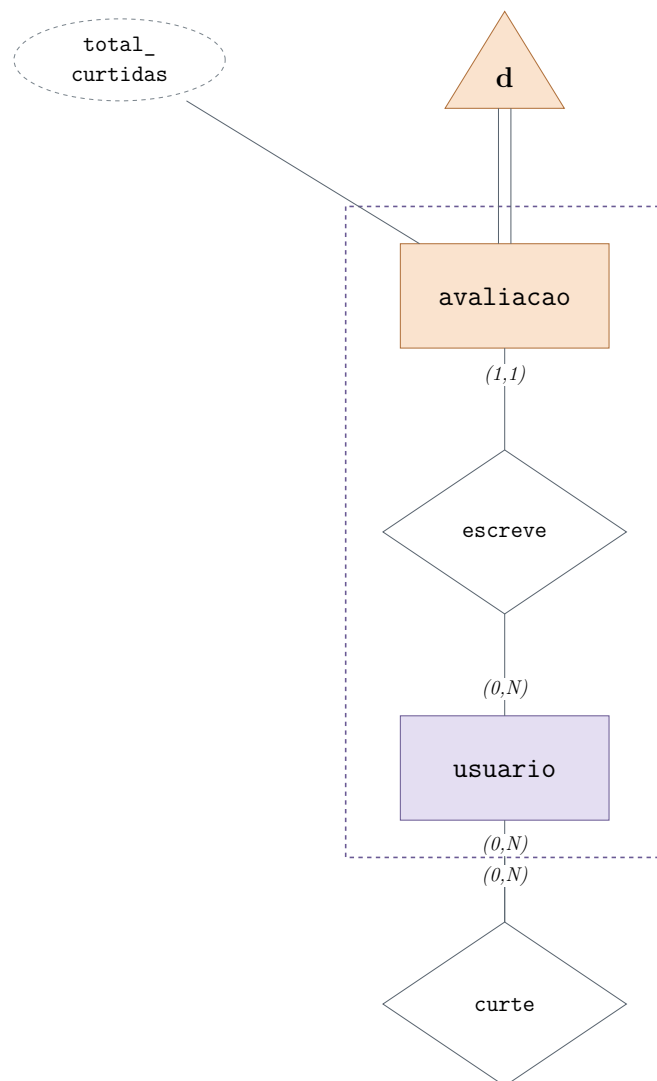


Figura 1.4: Ampliação da agregação. O retângulo tracejado envolve **usuario**, **escreve** e **avaliacao**; o **curte** parte da borda da caixa, e não de uma das entidades isoladamente. À esquerda, **total_curtidas** em elipse tracejada, o atributo derivado.

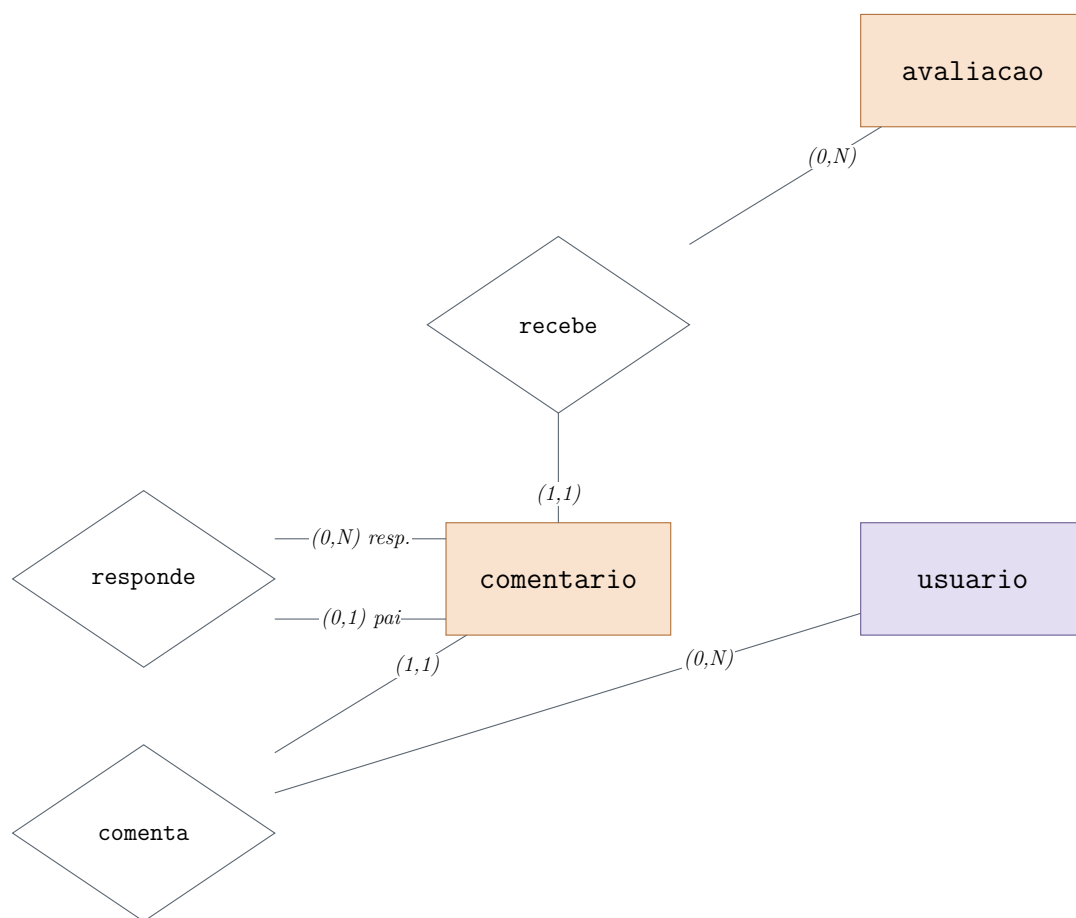


Figura 1.5: Ampliação da camada social, com o auto-relacionamento **responde** e seus papéis.

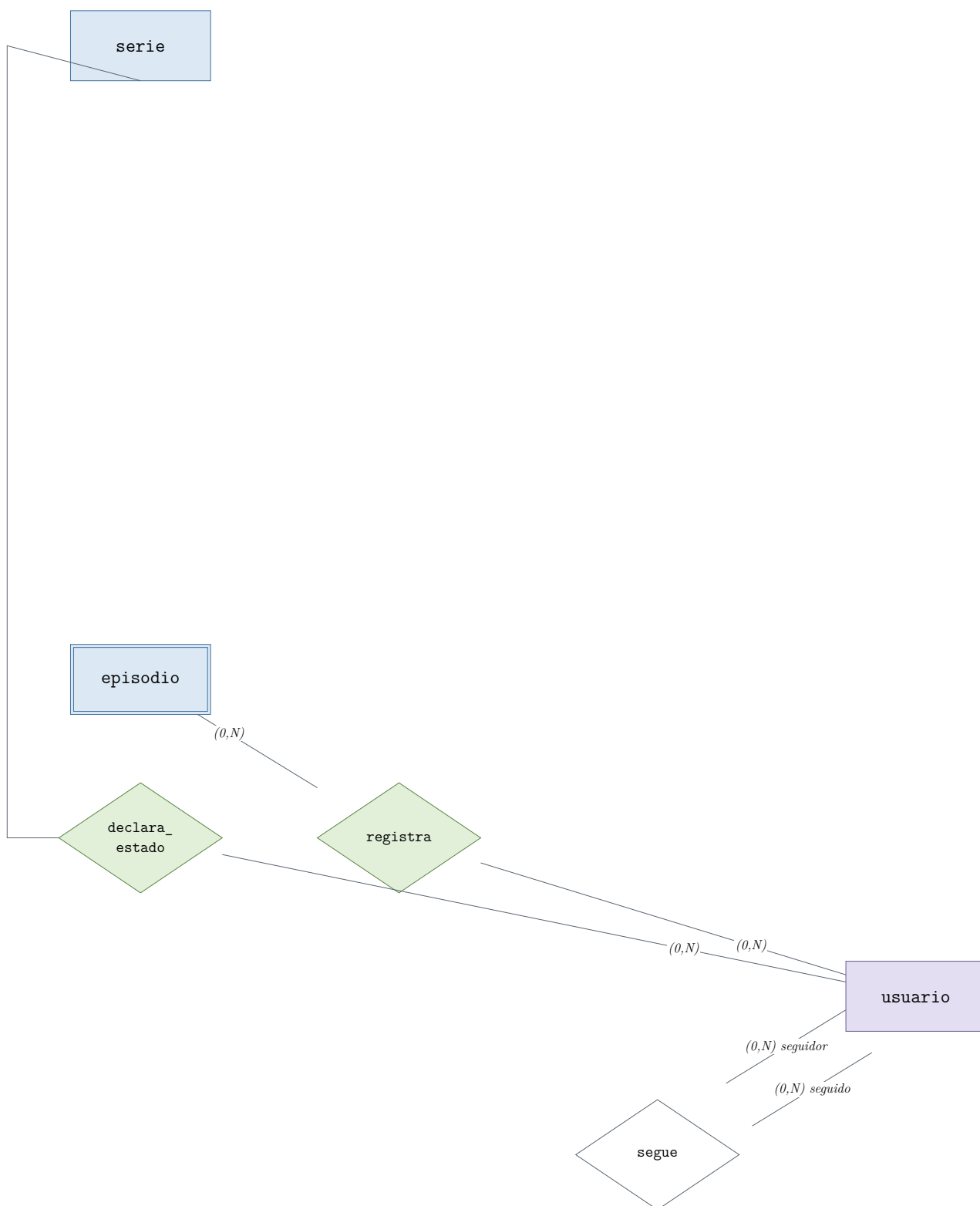


Figura 1.6: Ampliação do acompanhamento. As duas faces descritas na Seção 1.4 chegam a níveis diferentes da hierarquia: **declara_estado** na série, **registra** no episódio. À direita, o auto-relacionamento **segue** com os papéis nomeados.

Capítulo 2

Modelagem Lógica

Este capítulo converte o modelo conceitual do Capítulo 1 em um esquema relacional e demonstra que o resultado está normalizado. As seções seguem a ordem do raciocínio: as regras de conversão, o esquema que elas produzem e a verificação de que ele está normalizado. Nenhum nome definido no Capítulo 1 foi alterado aqui; as únicas construções novas são as relações que nascem de relacionamentos muitos-para-muitos, cujos nomes são anunciados na Seção 2.2.

2.1 Regras de mapeamento aplicadas

Orientação: *Enunciem cada regra de conversão do modelo ER para o relacional e mostrem onde ela foi aplicada. Não basta apresentar o esquema pronto: o professor avalia o processo de mapeamento (item 6.1 do conteúdo). A tabela abaixo já está organizada por tipo de construção; preencham a coluna da direita.*

O mapeamento seguiu as nove regras da Tabela 2.1, aplicadas nesta ordem: primeiro as entidades fortes, depois as fracas (que dependem das chaves das primeiras), depois os relacionamentos e, por último, a especialização.

Tabela 2.1: Regras de mapeamento do modelo ER para o modelo relacional

Construção no ER	Regra aplicada	Onde ocorre
Entidade forte	Torna-se uma relação. Atributos simples viram colunas e o identificador vira chave primária.	<code>usuario</code> , <code>serie</code> , <code>genero</code> , <code>pessoa</code> , <code>avaliacao</code> , <code>comentario</code>
Entidade fraca	Torna-se uma relação que recebe a chave do proprietário como chave estrangeira. Adotou-se chave substituta, com a chave parcial preservada por <code>UNIQUE(proprietário, numero)</code> .	<code>temporada</code> , <code>episodio</code>
Relacionamento 1:N	A relação do lado (1,1) recebe, como chave estrangeira, a chave primária do lado (1,N). Atributos próprios do relacionamento, se houvesse, iriam junto com a chave estrangeira.	<code>possui_temporada</code> , <code>possui_episodio</code> , <code>escreve</code> , <code>recebe</code> , <code>comenta</code> , <code>responde</code>
Relacionamento N:N sem atributo	Gera relação própria, cuja chave primária é a concatenação das chaves das duas entidades participantes.	<code>classifica</code> → <code>serie_genero</code>
Relacionamento N:N com atributo	Idem, acrescida dos atributos próprios do relacionamento como colunas não-chave, exceto quando o atributo precisa integrar a chave, como em <code>participacao</code> e <code>registro</code> .	<code>participa</code> → <code>participacao</code> ; <code>declara_estado</code> → <code>estado_serie</code> ; <code>registra</code> → <code>registro</code> ; <code>curte</code> → <code>curtida</code>
Auto-relacionamento N:N	Gera relação com duas chaves estrangeiras para a mesma tabela, renomeadas segundo os papéis do diagrama.	<code>segue(id_seguidor, id_seguido)</code>
Auto-relacionamento 1:N	Não gera relação: vira chave estrangeira recursiva e opcional na própria relação.	<code>responde</code> → <code>comentario.id_comentario_pai</code>
Especialização total e disjunta	Estratégia (b): uma relação para a superclasse e uma por subclasse, cuja chave primária é também chave estrangeira para a superclasse. Ver Seção 2.1.1.	<code>avaliacao</code> e suas três subclasses
Atributo derivado	Materializado como coluna, acompanhado do gatilho que o mantém igual à contagem real (Seção 3.4).	<code>avaliacao.total_curtidas</code>

O mapeamento tem limites, e vale registrar quais.

Participações mínimas iguais a 1 do lado N se perdem. O modelo relacional expressa participação total apenas do lado que carrega a chave estrangeira, tornando-a NOT NULL: é assim que “toda temporada pertence a uma série” (RN06) sobrevive, em `temporada.id_serie` NOT NULL. A recíproca não tem tradução declarativa. Para exigir que toda série tenha ao menos uma temporada, ou que toda série pertença a pelo menos um gênero (RN05), seria preciso verificar, a cada inserção em `serie`, uma linha que ainda não existe em `serie_genero`. Essas duas restrições migram para gatilho postergado ou para a camada de aplicação, e estão assinaladas como tal na tabela de rastreabilidade (Seção 3.8).

A agregação não possui regra de mapeamento própria, o que não a torna irrelevante: ela já cumpriu seu papel no modelo conceitual, ao determinar que `curte` e `recebe` partem do relacionamento reificado. Como `avaliacao` foi mapeada para uma relação com chave primária própria, os relacionamentos que saem da agregação se convertem como quaisquer outros: `curte` é um N:N comum entre `usuario` e `avaliacao`, e `recebe` é um 1:N comum. A agregação reaparece na política de exclusão, no ON DELETE CASCADE a partir de `avaliacao` (RN24), que implementa a dependência existencial descrita na Seção 1.6.

Já as regras que comparam colunas de tabelas diferentes não sobrevivem como restrição. RN18 (só se avalia episódio já registrado), RN19 (uma avaliação por usuário por alvo), RN22 (ninguém curte a própria avaliação) e RN14 (registrar promove o estado) envolvem duas relações simultaneamente, e um CHECK do PostgreSQL não pode consultar outra tabela. Todas foram mapeadas para os gatilhos da Tabela 3.1. A Seção 2.3 mostra, no caso da RN19, de onde vem essa limitação: o autor está na superclasse e o alvo na subclasse, de modo que nenhuma chave candidata alcança as duas colunas.

2.1.1 Tratamento da especialização

Orientação: Esta subseção é obrigatória. Existem três estratégias clássicas para mapear especialização: (a) tabela única com atributos nulos e discriminador; (b) uma tabela por subclasse, ligada à superclasse; (c) uma tabela por subclasse concreta, sem tabela para a superclasse. Apresentem as três, digam qual foi adotada e por quê. A estratégia escolhida no projeto é a (b) – justifiquem-na considerando que (i) a superclasse possui atributos próprios relevantes (`nota`, `texto`, `data_publicacao`), (ii) curtidas e comentários precisam referenciar a avaliação independentemente do tipo de alvo, o que exige uma chave única compartilhada, e (iii) a estratégia (c) tornaria impossível a referência da agregação descrita na Seção 1.6.

A especialização de `avaliacao` (Seção 1.5) admite três estratégias de mapeamento, resumidas na Tabela 2.2.

Tabela 2.2: Estratégias de mapeamento da especialização

Estratégia	Como ficaria	Por que foi ou não adotada
(a) Tabela única	Uma só relação <code>avaliacao</code> , com as colunas <code>id_serie</code> , <code>id_temporada</code> e <code>id_episodio</code> todas opcionais e um discriminador <code>tipo_alvo</code> .	Descartada. Duas das três colunas de ligação seriam sempre nulas, e a exclusividade dependeria de um <code>CHECK</code> que conta colunas não nulas, uma regra externa ao modelo.
(b) Uma tabela por subclasse, ligada à superclasse	<code>avaliacao</code> guarda o que é comum; <code>avaliacao_serie</code> , <code>avaliacao_temporada</code> e <code>avaliacao_episodio</code> guardam cada uma a ligação com seu alvo, e sua chave primária é também chave estrangeira para <code>avaliacao</code> .	Adotada. Nenhuma coluna nula por construção, e a chave compartilhada permite que <code>curtida</code> e <code>comentario</code> referenciem a avaliação sem saber o tipo do alvo.
(c) Uma tabela por subclasse concreta	Três relações independentes, cada uma repetindo <code>nota</code> , <code>texto</code> , <code>contem_spoiler</code> , <code>data_publicacao</code> e <code>id_usuario</code> ; nenhuma relação para a superclasse.	Descartada. Sem chave compartilhada, <code>curtida</code> e <code>comentario</code> precisariam de três chaves estrangeiras opcionais cada, reintroduzindo uma camada adiante o problema que (a) causava.

A estratégia (b) foi adotada por três razões, na ordem de peso.

1. A superclasse tem substância própria: `nota`, `texto`, `contem_spoiler` e `data_publicacao` pertencem ao ato de avaliar, e não ao alvo. Suprimir a relação da superclasse, como faz (c), obrigaria a repetir esses quatro atributos três vezes, e qualquer consulta sobre “avaliações” (o ranking da Consulta 1, por exemplo) teria de unir três relações antes de começar.
2. Curtidas e comentários precisam de uma chave única compartilhada. `curtida` e `comentario` referenciam a avaliação sem se importar com o tipo do alvo, e com (b) a chave estrangeira aponta para `avaliacao` e pronto. Com (c), `id_avaliacao` não seria único entre as três relações, e cada referência precisaria de três colunas opcionais e de uma restrição para garantir que apenas uma estivesse preenchida.
3. A agregação exige um referente único. A Seção 1.6 estabelece que curtir incide sobre a associação inteira, e uma associação sem identidade própria no esquema não pode ser referenciada, que é o que (c) produziria. Com (b), essa identidade existe em `avaliacao.id_avaliacao`.

O custo da escolha é que a disjunção deixa de ser estrutural. Nada, no esquema, impede que o mesmo `id_avaliacao` apareça ao mesmo tempo em `avaliacao_serie` e em `avaliacao_episodio`, caso em que o discriminador `tipo_alvo` diria uma coisa e as tabelas, outra. Há duas saídas. A primeira é acrescentar `tipo_alvo` a cada subclasse, fixá-lo por `CHECK` e referenciar `avaliacao(id_avaliacao, tipo_alvo)` por chave estrangeira composta, o que torna a disjunção declarativa ao preço de uma coluna redundante em três tabelas. A segunda é verificá-la por gatilho. Optou-se pela segunda, para não introduzir colunas ausentes do dicionário de dados do Capítulo 1. A totalidade também é inexprimível de forma declarativa e recai na aplicação: a inserção em `avaliacao` e na subclasse correspondente deve ocorrer na mesma transação.

2.1.2 Tratamento das entidades fracas

Orientação: Expliquem a diferença entre a chave da entidade fraca no modelo conceitual (chave parcial + chave do proprietário) e a solução adotada no esquema físico. No projeto, adotou-se chave primária substituta (*id_temporada*, *id_episodio*) acompanhada de restrição de unicidade sobre o par (proprietário, número). Justifiquem: a chave composta natural funcionaria, mas se propagaria para todas as tabelas que referenciam episódio – *registro* e *avaliacao_episodio* – aumentando o tamanho das chaves estrangeiras sem ganho semântico. A restrição **UNIQUE** preserva integralmente a regra original (RN06 e RN07).

No modelo conceitual, a identificação de uma entidade fraca é composta pela chave parcial mais a chave do proprietário. Aplicada literalmente, a regra produziria:

- *temporada*: chave primária (*id_serie*, *numero*);
- *episodio*: chave primária (*id_serie*, *numero_temporada*, *numero_episodio*), três colunas, porque a chave da temporada já é composta e se propaga.

Essa solução funciona e é semanticamente exata, mas a chave de *episodio* teria de ser copiada inteira para todas as relações que o referenciam: *registro* e *avaliacao_episodio*. Como *registro* é a maior tabela do sistema, com um evento por episódio assistido por usuário, o custo apareceria em cada linha e em cada índice, sem ganho semântico: ninguém consulta um registro de exibição pelo número da temporada.

Adotou-se, portanto, **chave primária substituta** (*id_temporada*, *id_episodio*) acompanhada de restrição de unicidade sobre a chave natural:

- *temporada*: **UNIQUE**(*id_serie*, *numero*), pela RN06;
- *episodio*: **UNIQUE**(*id_temporada*, *numero*), pela RN07.

A restrição **UNIQUE** preserva integralmente a regra original: continua sendo impossível cadastrar duas temporadas 2 na mesma série, ou dois episódios 5 na mesma temporada. A chave natural não desaparece: ela permanece como chave candidata, como registra a Seção 2.3, e apenas deixa de ser a chave escolhida para propagação. A dependência existencial da entidade fraca é garantida pela combinação de NOT NULL na chave estrangeira com ON DELETE CASCADE (RN08).

2.2 Esquema relacional

Orientação: Apresentem o esquema na notação $R(\text{chave}, \text{atributo}, \dots)$, com chaves estrangeiras indicadas. São 17 relações. Os nomes de tabela e de coluna estão fixados no Capítulo 1 e não devem ser alterados sem atualizar aquele capítulo. A lista abaixo enumera as relações esperadas, na ordem em que devem ser apresentadas.

O mapeamento produz **17 relações**, apresentadas abaixo na ordem de dependência: nenhuma relação referencia outra que apareça depois dela, de modo que a mesma ordem serve para os comandos **CREATE TABLE** do Capítulo 3 e para a carga de dados. A chave primária vem sublinhada; FK indica chave estrangeira e UK indica restrição de unicidade.

1. *usuario*(*id_usuario*, *username*, *email*, *nome_exibicao*, *bio*, *data_cadastro*)
UK: *username*; *email* (RN01)
2. *segue*(*id_seguidor*, *id_seguido*, *data_inicio*)
FK: *id_seguidor* → *usuario*; *id_seguido* → *usuario* (RN02, RN03)
3. *serie*(*id_serie*, *titulo*, *titulo_original*, *sinopse*, *ano_estreia*, *situacao*, *pais_origem*)

4. temporada(id_temporada, id_serie, numero, titulo, data_estreia)
FK: id_serie → serie UK: (id_serie, numero) (RN06, RN08)
5. episodio(id_episodio, id_temporada, numero, titulo, duracao_min, data_exibicao, sinopse)
FK: id_temporada → temporada UK: (id_temporada, numero) (RN07, RN08)
6. genero(id_genero, nome)
UK: nome
7. serie_genero(id_serie, id_genero)
FK: id_serie → serie; id_genero → genero (RN05)
8. pessoa(id_pessoa, nome, data_nascimento, nacionalidade)
9. participacao(id_pessoa, id_serie, funcao, personagem)
FK: id_pessoa → pessoa; id_serie → serie (RN09, RN10)
10. estado_serie(id_usuario, id_serie, estado, data_atualizacao)
FK: id_usuario → usuario; id_serie → serie (RN13, RN14)
11. registro(id_registro, id_usuario, id_episodio, data_assistido)
FK: id_usuario → usuario; id_episodio → episodio
UK: (id_usuario, id_episodio, data_assistido) (RN11, RN12)
12. avaliacao(id_avaliacao, id_usuario, nota, texto, contem_spoiler, data_publicacao, tipo_alvo, total_curtidas)
FK: id_usuario → usuario (RN15–RN17, RN20)
13. avaliacao_serie(id_avaliacao, id_serie)
FK: id_avaliacao → avaliacao; id_serie → serie
14. avaliacao_temporada(id_avaliacao, id_temporada)
FK: id_avaliacao → avaliacao; id_temporada → temporada
15. avaliacao_episodio(id_avaliacao, id_episodio)
FK: id_avaliacao → avaliacao; id_episodio → episodio (RN18)
16. curtida(id_usuario, id_avaliacao, data_curtida)
FK: id_usuario → usuario; id_avaliacao → avaliacao (RN21, RN22)
17. comentario(id_comentario, id_avaliacao, id_usuario, id_comentario_pai, texto, data_publicacao)
FK: id_avaliacao → avaliacao; id_usuario → usuario; id_comentario_pai → comentario (RN23, RN24)

Algumas escolhas de chave merecem registro.

Em *participacao* a chave tem três colunas. A RN09 permite que a mesma pessoa exerça mais de uma função na mesma série, o que impede que a chave seja apenas (*id_pessoa*, *id_serie*): *funcao* precisa integrá-la. Isso torna *personagem* o único atributo não-primo da relação, e é justamente ele que a RN10 restringe, exigindo valor não nulo quando *funcao* = *ator* e nulo nas demais funções. O esquema assume, como simplificação, que uma pessoa interpreta no máximo um personagem por série; para admitir atores em papéis múltiplos, *personagem* passaria a integrar a chave primária.

registro é a única relação derivada de um N:N cuja chave não é o par de chaves estrangeiras, por causa da RN11: o mesmo usuário pode registrar o mesmo episódio em datas diferentes, de modo que (**id_usuario**, **id_episodio**) não identifica uma linha. A chave natural passa a ser a tripla que inclui **data_assistido**, declarada como **UNIQUE**, o que impede o registro acidentalmente duplicado no mesmo dia sem impedir a reexibição.

Em **estado_serie**, o par (**id_usuario**, **id_serie**) como chave primária é o próprio enunciado da RN13, “no máximo um estado por usuário por série”. A regra fica implementada pela chave, sem restrição adicional. Séries sobre as quais o usuário nada declarou não possuem linha, em vez de uma linha com estado nulo.

Nas três subclasses, a chave primária é também chave estrangeira. Ela é herdada da superclasse, o que garante a correspondência um-para-um: uma subclasse não pode ter duas linhas para a mesma avaliação, porque a chave primária o proíbe.

A Tabela 2.3 reúne as políticas de exclusão adotadas para as chaves estrangeiras, que o Capítulo 3 detalha no DDL. O critério é único: **CASCADE** onde o modelo conceitual declara dependência existencial, **RESTRICT** onde a exclusão representaria perda de informação do acervo.

Tabela 2.3: Políticas de integridade referencial

Chave estrangeira	Política	Justificativa
Todas as que referenciam usuario	CASCADE	RN04: excluir o usuário elimina todo o seu histórico.
temporada.id_serie ,	CASCADE	RN08: dependência existencial da entidade fraca.
episodio.id_temporada		
curtida.id_avaliacao ,	CASCADE	RN24: a curtida e o comentário perdem o objeto.
comentario.id_avaliacao		
comentario.id_comentario_pai	CASCADE	RN23: excluir o comentário-pai elimina a subárvore de respostas.
As três subclasses → avaliacao	CASCADE	A subclasse não existe sem a superclasse.
serie_genero , participacao → acervo	CASCADE	Classificação e elenco pertencem à série.
Subclasses → alvo do acervo ; registro.id_episodio ; estado_serie.id_serie	RESTRICT	Excluir um episódio avaliado ou assistido apagaria silenciosamente histórico de usuários.

2.3 Dependências funcionais

Orientação: Para cada relação, listem as dependências funcionais na forma $X \rightarrow Y$ e identifiquem as chaves candidatas. Esta seção é o insumo da seção seguinte: sem as dependências explícitas, a demonstração de normalização fica sem base. A tabela abaixo já traz as relações; preencham as duas colunas restantes. Atenção especial às relações de chave composta – **participacao**, **estado_serie**, **serie_genero**, **segue** e **curtida** – que são onde uma violação de 2FN apareceria, caso existisse.

Na tabela a seguir, o símbolo * após um atributo indica “e todos os demais atributos da relação”. Chaves candidatas em negrito são as escolhidas como chave primária.

Relação	Dependências funcionais	Chaves candidatas
usuario	id_usuario → * username → * email → *	id_usuario ; username; email

Relação	Dependências funcionais	Chaves candidatas
segue	$(id_seguidor, id_seguido) \rightarrow data_inicio$	$(id_seguidor, id_seguido)$
serie	$id_serie \rightarrow *$	id_serie
temporada	$id_temporada \rightarrow *$ $(id_serie, numero) \rightarrow *$	$id_temporada;$ $(id_serie, numero)$
episodio	$id_episodio \rightarrow *$ $(id_temporada, numero) \rightarrow *$	$id_episodio;$ $(id_temporada, numero)$
genero	$id_genero \rightarrow nome$ $nome \rightarrow id_genero$	$id_genero; nome$
serie_genero	Nenhuma dependência não trivial: a relação é toda chave	(id_serie, id_genero)
pessoa	$id_pessoa \rightarrow *$	id_pessoa
participacao	$(id_pessoa, id_serie, funcao) \rightarrow personagem$	$(id_pessoa, id_serie, funcao)$
estado_serie	$(id_usuario, id_serie) \rightarrow estado, data_atualizacao$	$(id_usuario, id_serie)$
registro	$id_registro \rightarrow *$ $(id_usuario, id_episodio, data_assistido) \rightarrow id_registro$	$id_registro;$ $(id_usuario, id_episodio, data_assistido)$
avaliacao	$id_avaliacao \rightarrow *$	$id_avaliacao$
avaliacao_serie	$id_avaliacao \rightarrow id_serie$	$id_avaliacao$
avaliacao_temporada	$id_avaliacao \rightarrow id_temporada$	$id_avaliacao$
avaliacao_episodio	$id_avaliacao \rightarrow id_episodio$	$id_avaliacao$
curtida	$(id_usuario, id_avaliacao) \rightarrow data_curtida$	$(id_usuario, id_avaliacao)$
comentario	$id_comentario \rightarrow *$	$id_comentario$

As relações de chave composta não escondem dependência parcial. Em *segue*, *serie_genero*, *estado_serie* e *curtida*, todo atributo não-primo (*data_inicio*, *estado*, *data_atualizacao*, *data_curtida*) depende da chave *inteira*: a data em que *A* passou a seguir *B* é propriedade do par, e não de *A* ou de *B* isoladamente. Em *participacao*, *personagem* depende das três colunas, porque o *personagem* é função de quem, em qual série e em que papel. É essa verificação que garante a 2FN dessas cinco relações.

As chaves naturais sobreviveram como chaves candidatas. As duas entidades fracas e *registro* têm duas chaves candidatas cada: a substituta, escolhida para propagação, e a natural, mantida por **UNIQUE**. Nenhuma informação de identificação se perdeu na troca.

A RN19, por outro lado, não tem como ser chave. As três subclasses determinam apenas o alvo, porque *id_usuario* vive na superclasse. Para que “um usuário possui no máximo uma avaliação por alvo” fosse uma restrição de chave, seria preciso uma unicidade sobre (*avaliacao.id_usuario*, *avaliacao_serie.id_serie*), duas colunas em duas relações diferentes, o que nenhuma restrição declarativa alcança. Daí o gatilho

`trg_avaliacao_unica_por_alvo`, que é o preço da estratégia (b); com a estratégia (a), a mesma regra seria um `UNIQUE`.

Resta `total_curtidas`, que é redundância controlada. As formas normais se definem sobre dependências *dentro* de uma relação, e `total_curtidas` não é determinado por nenhum atributo não-primo de `avaliacao`: ele vem de uma consulta de agregação sobre `curtida`. Formalmente, portanto, `avaliacao` permanece em 3FN. A redundância é real, mas interrelacional, e a contrapartida assumida na Seção 1.7 é o gatilho `trg_atualiza_contador_curtidas`, que mantém coluna e contagem sincronizadas.

2.4 Normalização

Orientação: O esquema apresentado na Seção 2.2 já nasce na 3FN, porque foi obtido por mapeamento de um modelo ER bem construído. Isso cria um problema de apresentação: não há nada para normalizar, e a seção corre o risco de virar a afirmação vazia “está tudo normalizado”.

A forma recomendada de resolver isso é **partir de uma relação universal propositalmente mal projetada** e decompô-la passo a passo até chegar ao esquema final, mostrando em cada passo qual anomalia é eliminada. Sugestão de relação de partida, que concentra os três problemas clássicos:

`AVALIACAO_COMPLETA(id_usuario, id_episodio, username, email, titulo_serie, ano_estreia, numero_temporada, numero_episodio, titulo_episodio, duracao_min, nota, texto, data_publicacao, generos)`

Roteiro sugerido:

- **1FN** – o atributo `generos` é multivalorado (uma série tem vários gêneros). Mostrem a anomalia concreta: impossibilidade de consultar “séries de drama” sem varrer `texto`. A decomposição gera `genero` e `serie_genero`.
- **2FN** – com chave composta (`id_usuario`, `id_episodio`), os atributos `username` e `email` dependem apenas de `id_usuario`, e `titulo_episodio` e `duracao_min` dependem apenas de `id_episodio`: são dependências parciais. Mostrem a anomalia de atualização (trocar o e-mail exige alterar todas as avaliações do usuário) e a de exclusão (apagar a última avaliação apagaria o usuário). A decomposição gera `usuario` e `episodio`.
- **3FN** – `titulo_serie` e `ano_estreia` dependem de `numero_temporada`, que por sua vez depende do episódio: dependência transitiva. A decomposição gera `serie` e `temporada`, completando a hierarquia.

Fechem a seção afirmando que o esquema final está em 3FN e verificando, relação a relação, que nenhum atributo não-primo depende parcial ou transitivamente da chave. Se decidirem discutir BCNF, verifiquem que todo determinante é chave candidata – no esquema proposto isso se verifica, pois não há relações com duas chaves candidatas sobrepostas.

O esquema da Seção 2.2 já nasce na 3FN, porque foi obtido por mapeamento de um modelo ER em que cada conceito do minimundo virou uma entidade própria. Afirmar isso, porém, não demonstra nada. Para tornar a normalização verificável, esta seção parte de uma relação universal propositalmente mal projetada, uma única tabela que guardasse “tudo sobre uma avaliação de episódio”, e a decompõe passo a passo, exibindo, em cada etapa, a anomalia eliminada.

A relação de partida é:

```
AVALIACAO_COMPLETA(id_usuario, id_episodio, username, email, titulo_serie,
ano_estreia, numero_temporada, numero_episodio, titulo_episodio,
duracao_min, nota, texto, data_publicacao, generos)
```

com chave (id_usuario, id_episodio), já que um usuário avalia um episódio uma única vez (RN19).

2.4.1 Primeira forma normal

O atributo **generos** viola a 1FN: uma série pertence a vários gêneros (RN05), e a única forma de acomodá-los em uma coluna é como lista, "Drama, Suspense, Ficcao Cientifica".

A anomalia é imediata. A consulta “quais séries são de drama” deixa de ser uma junção e vira uma busca textual, sujeita a erro: `LIKE '%Drama%'` também casa com um gênero hipotético “Dramédia”. Pior, não existe forma de garantir que “Ficção Científica” e “Ficcao Cientifica” sejam o mesmo gênero, porque não há entidade que os represente, e o vocabulário de gêneros passa a ser uma convenção de digitação. Renomear um gênero exigiria reescrever a lista em todas as linhas de todas as avaliações de todas as séries daquele gênero.

A decomposição extrai o atributo multivalorado para uma relação própria, com a entidade que lhe dá vocabulário controlado:

```
genero(id_genero, nome)
serie_genero(id_serie, id_genero)
```

serie_genero é uma relação toda chave: não há atributo não-primo, e portanto ela é trivialmente 2FN, 3FN e BCNF. **genero** tem duas chaves candidatas (id_genero e nome), e é a unicidade de nome que torna impossível cadastrar o mesmo gênero duas vezes com grafias diferentes.

2.4.2 Segunda forma normal

Com a chave composta (id_usuario, id_episodio), vários atributos dependem de apenas metade dela:

- id_usuario → username, email;
- id_episodio → numero_episodio, titulo_episodio, duracao_min.

São dependências parciais, e cada uma produz seu conjunto de anomalias.

Anomalia de atualização. O e-mail de um usuário está repetido em cada avaliação que ele publicou. Alterá-lo exige um `UPDATE` em todas as linhas; se uma escapar, o banco passa a afirmar dois e-mails para a mesma pessoa, e não há critério interno para decidir qual é o verdadeiro.

Anomalia de exclusão. Um usuário que apaga sua única avaliação desaparece do sistema: seu username e seu e-mail sumiriam junto com a linha, embora a conta deva continuar existindo. O mesmo vale pelo outro lado: apagar a última avaliação de um episódio apagaria a duração e o título daquele episódio, que são dados do acervo e não do usuário.

Anomalia de inserção. Um usuário recém-cadastrado que ainda não avaliou nada não pode ser representado, porque não existe linha sem id_episodio. Um episódio recém-lançado, idem. A relação universal só admite existir quem já interagiu.

A decomposição separa cada lado da chave em sua própria relação:

```

usuario(id_usuario, username, email, ...)
episodio(id_episodio, ..., numero, titulo, duracao_min, ...)
avaliacao(id_avaliacao, id_usuario, nota, texto, data_publicacao,
...)
```

Note-se que a decomposição é também o que permite atender à RN01: com `usuario` isolada, `UNIQUE(email)` passa a ser verificável; na relação universal, a mesma restrição seria impossível, já que o e-mail se repete legitimamente em cada avaliação do usuário.

2.4.3 Terceira forma normal

Restam em `AVALIACAO_COMPLETA` os atributos `titulo_serie`, `ano_estreia` e `numero_temporada`. Eles dependem de `id_episodio`, que é metade da chave, e a dependência é *transitiva*:

$$\text{id_episodio} \rightarrow \text{numero_temporada} \rightarrow \text{titulo_serie}, \text{ano_estreia}$$

O título da série pertence à série, e chega ao episódio através da temporada.

Corrigir o título de uma série obriga, nesse desenho, a percorrer todos os seus episódios, em todas as temporadas. Como a hierarquia tem três níveis, o mesmo título pode estar replicado em dezenas de linhas por temporada. E uma série recém-cadastrada, ainda sem episódios lançados, novamente não teria onde existir.

A decomposição quebra a cadeia transitiva em seus três elos, o que reconstrói a hierarquia do minimundo:

```

serie(id_serie, titulo, ano_estreia, ...)
temporada(id_temporada, id_serie, numero, ...)
episodio(id_episodio, id_temporada, numero, ...)
```

Cada atributo passa a residir no nível a que pertence, e cada nível guarda apenas a referência ao nível imediatamente superior. A hierarquia de três níveis do Capítulo 1 é, portanto, uma exigência de normalização: qualquer tentativa de achatá-la reintroduz a dependência transitiva.

2.4.4 Verificação do esquema final

Percorrendo as 17 relações da Seção 2.2 com as dependências funcionais da Seção 2.3:

- **1FN**: todos os atributos são atômicos. Nenhuma coluna guarda lista, e as duas construções que poderiam tentá-lo, gêneros de uma série e funções de uma pessoa em uma série, foram mapeadas para `serie_genero` e `participacao`.
- **2FN**: as relações de chave simples satisfazem a condição trivialmente. Das cinco de chave composta, `serie_genero` não tem atributo não-primo, e em `segue`, `estado_serie`, `curtida` e `participacao` os atributos não-primos dependem da chave inteira, conforme verificado ao final da Seção 2.3.
- **3FN**: não há dependência transitiva. Toda relação descreve um único conceito, e atributos de conceitos vizinhos aparecem apenas como chave estrangeira. O caso que mereceria dúvida, `total_curtidas`, foi analisado ao final da Seção 2.3: por ser derivado de outra relação, não configura dependência transitiva interna.
- **BCNF**: em toda relação, todo determinante é chave candidata. Basta percorrer a coluna central da tabela da Seção 2.3: não há um único lado esquerdo que não seja chave candidata, e não existem duas chaves candidatas compostas e sobrepostas, que é a situação capaz de produzir violação de BCNF sem violação de 3FN. O esquema está, portanto, em BCNF.

Vale observar que a decomposição é **sem perda de junção** e **preserva dependências**: cada passo separou a relação por um determinante que permaneceu como chave estrangeira no lado dependente, de modo que a junção pelas chaves reconstrói `AVALIACAO_COMPLETA` sem gerar linhas espúrias. A visão `vw_avaliacao_completa` do Capítulo 3 faz exatamente essa reconstrução: a redundância deixou de ser armazenada e passou a ser computada quando necessária.

2.5 Diagrama lógico

Orientação: Diagrama do esquema relacional, com todas as tabelas, colunas, chaves primárias e chaves estrangeiras – notação de “pé de galinha” ou equivalente. Pode ser gerado no `brModelo` (a partir do conceitual) ou por engenharia reversa a partir do banco criado, usando `pgAdmin` ou `DBeaver`. Salvem em `images/der-logico.png` e descomentem o bloco abaixo. Se usarem engenharia reversa, confirmem se o diagrama não ficou incompleto: ferramentas costumam omitir restrições `CHECK` e gatilhos, que então precisam ser mencionados no texto.

A Figura 2.1 apresenta as dezessete relações em notação de pé de galinha. Cada tabela é um retângulo com o nome no cabeçalho e uma linha por coluna. O marcador à esquerda de cada linha distingue chave primária (PK), chave estrangeira (FK), coluna que acumula as duas funções (PF) e restrição de unicidade (UK); as colunas que compõem a chave primária aparecem sublinhadas. Nas ligações, o pé de galinha marca o lado que admite muitas linhas e o traço simples marca o lado referenciado, que é sempre a chave primária da tabela de destino.

Como o esquema é denso, as Figuras 2.2 e 2.3 ampliam as duas regiões de maior encadeamento. O diagrama é, além de documentação, um instrumento de conferência: ele e o DDL do Capítulo 3 devem coincidir coluna a coluna, e qualquer divergência entre os dois indica erro em um deles.

A leitura acompanha os três pilares do minimundo. Ao centro fica a cadeia do acervo, **serie**, **temporada** e **episodio**, e logo abaixo dela as três subclasses da especialização. À esquerda ficam as tabelas que só existem em função de outras: as duas nascidas de relacionamentos muitos-para-muitos do acervo, **serie_genero** e **participacao**, e as duas do acompanhamento, **estado_serie** e **registro**, que são as duas faces descritas na Seção 1.4 e chegam a níveis diferentes da hierarquia: a intenção declarada na série, o histórico efetivo no episódio. À direita ficam **usuario** e **avaliacao** com a camada social, **segue**, **curtida** e **comentario**.

O encadeamento de chaves estrangeiras torna visível uma propriedade que o texto vinha afirmando: **usuario** e **avaliacao** concentram a maior parte das referências, e nenhuma tabela do acervo aponta para o usuário. O acervo existe independentemente de quem o assiste, e é o acompanhamento que faz a ponte entre os dois lados.

O diagrama deixa de fora duas coisas que precisam ser enunciadas aqui. As restrições `CHECK` não têm representação gráfica em nenhuma notação relacional: faixa e granularidade da nota (RN16), avaliação não vazia (RN17), data de exibição não futura (RN12), coerência entre **funcao** e **personagem** (RN10) e proibição de seguir a si mesmo (RN03). Os cinco gatilhos da Tabela 3.1 também não aparecem, e são eles que implementam as regras interrelacionais identificadas na Seção 2.1. Quem lesse apenas o diagrama concluiria que o esquema permite avaliar um episódio nunca assistido ou curtir a própria avaliação.

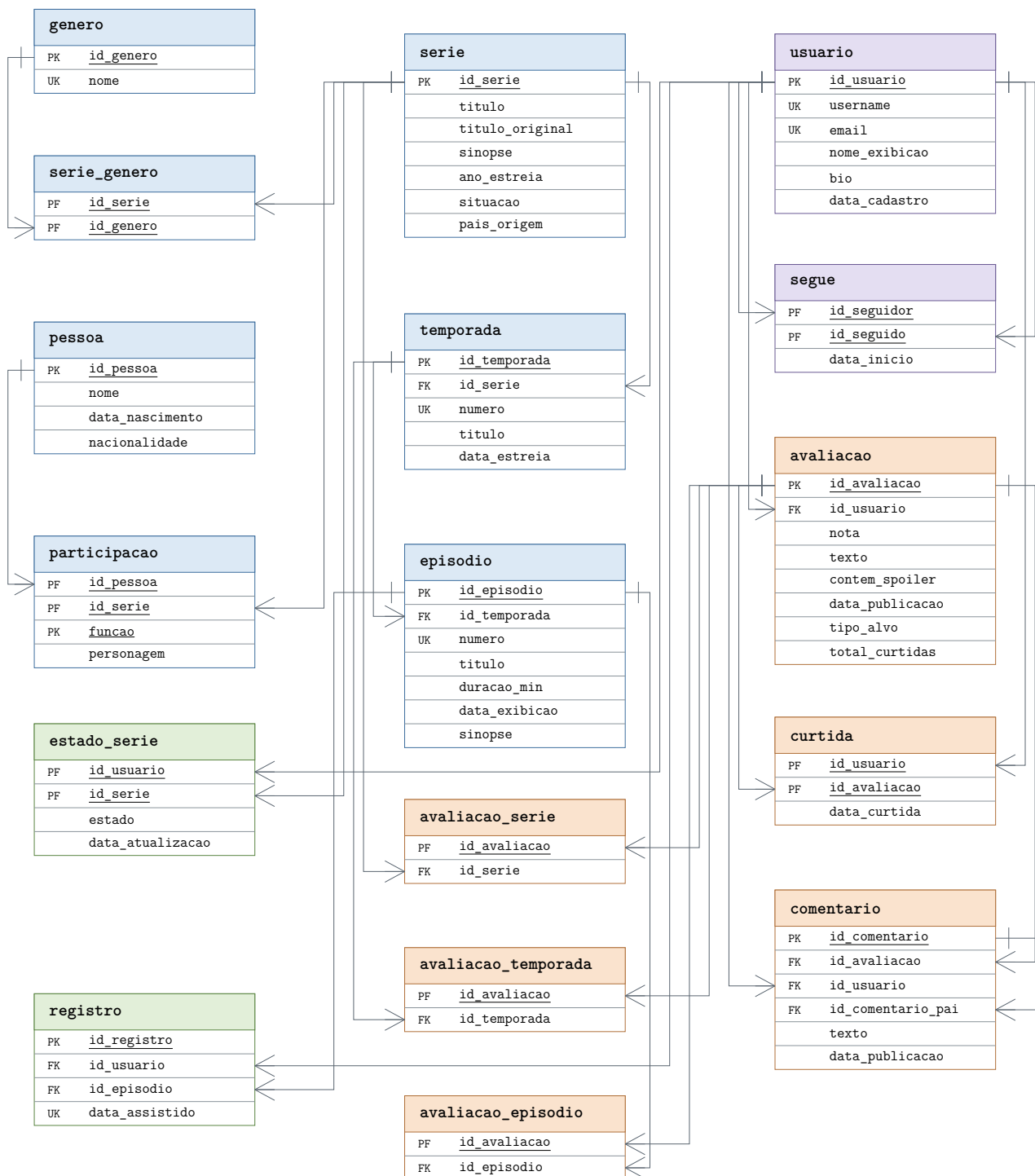


Figura 2.1: Diagrama lógico do banco de dados. As dezessete relações do esquema, com suas colunas, chaves primárias, chaves estrangeiras e restrições de unicidade. As cores acompanham os três pilares da Seção 1.4: acervo em azul, usuário em roxo, avaliação e camada social em âmbar, e as duas tabelas de acompanhamento em verde.

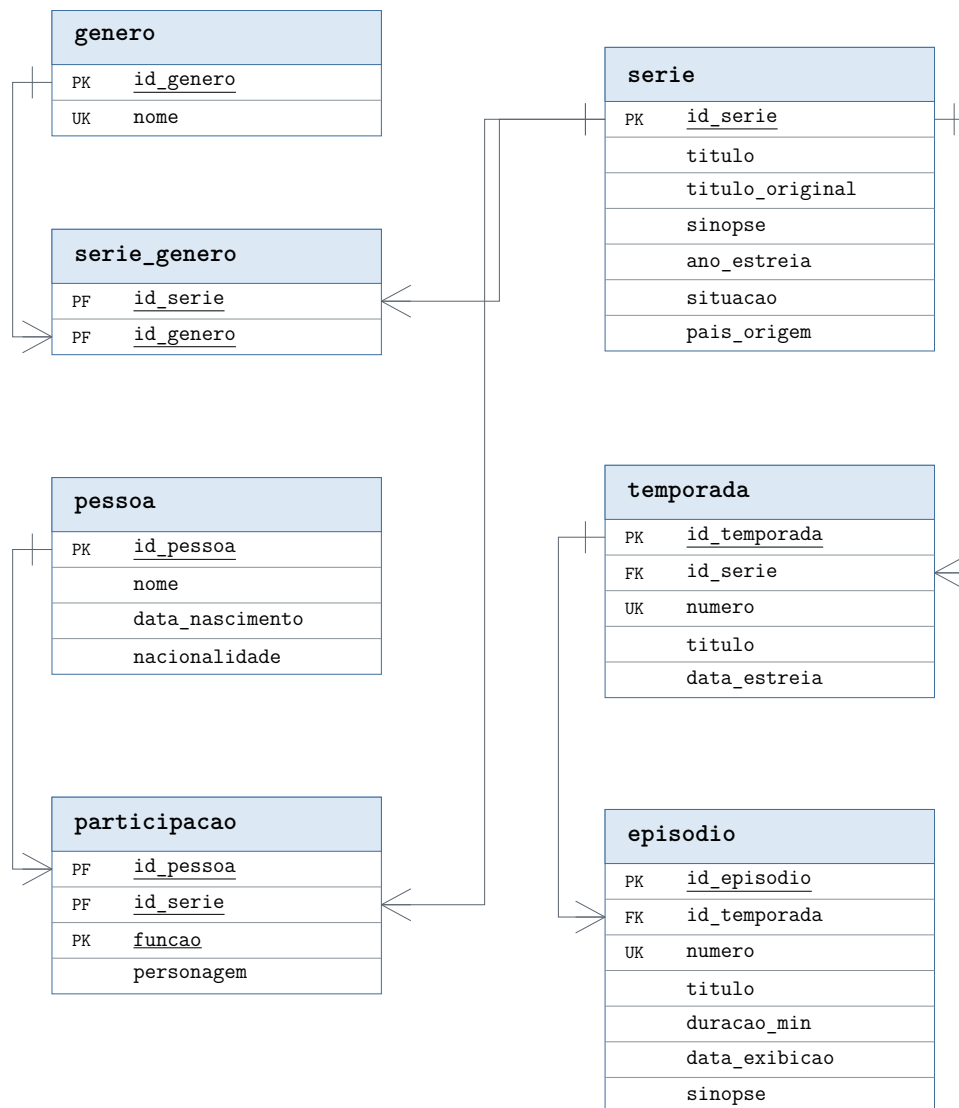


Figura 2.2: Ampliação do acervo. A cadeia **serie**, **temporada** e **episodio**, com a chave substituta de cada nível e a restrição **UNIQUE** sobre a chave natural, mais as duas tabelas nascidas de relacionamentos muitos-para-muitos.

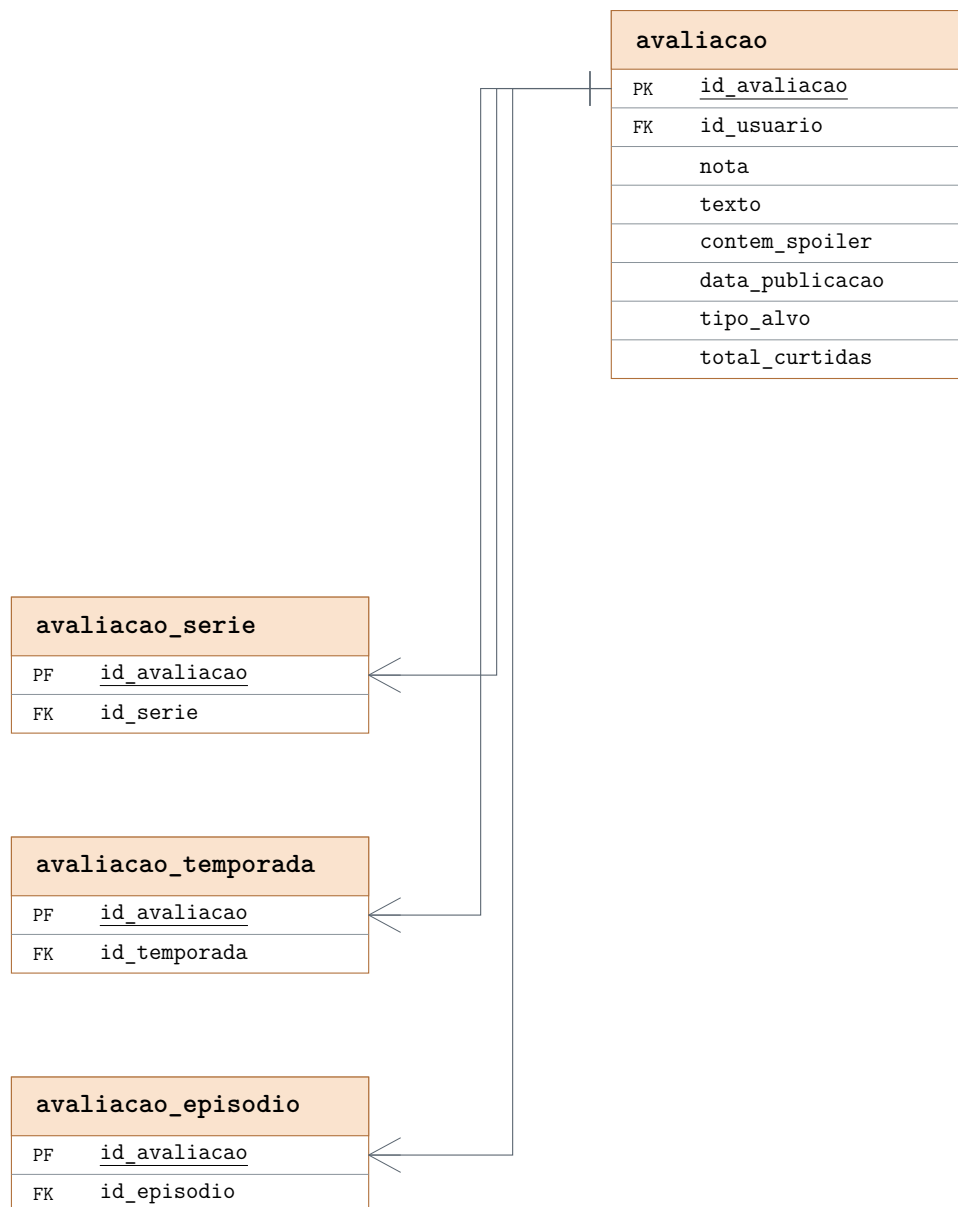


Figura 2.3: Ampliação da especialização. Nas três subclasses, `id_avaliacao` é ao mesmo tempo chave primária e chave estrangeira, o que impede que uma avaliação tenha duas linhas na mesma subclasse e produz a correspondência um-para-um com a superclasse.

Capítulo 3

Modelagem Física

***Orientação:** Capítulo a ser escrito. Contém o script SQL executável no PostgreSQL e sua explicação. O script completo vai no apêndice ou em arquivo anexo; neste capítulo entram os trechos que ilustram decisões, sempre acompanhados de explicação. Código sem texto explicativo não pontua.*

3.1 Convenções de nomenclatura e criação do esquema

***Orientação:** Registrem as convenções adotadas – elas já estão em uso no Capítulo 1 e devem ser seguidas: nomes de tabela no singular e em minúsculas; chave primária no formato `id_<tabela>`; `snake_case` para colunas compostas; objetos criados dentro de um esquema próprio, e não em `public`.*

```
DROP SCHEMA IF EXISTS serie_tracker CASCADE;  
CREATE SCHEMA serie_tracker;  
SET search_path TO serie_tracker;
```

Listing 3.1: Criação do esquema

3.2 Definição de dados (DDL)

***Orientação:** Apresentem os comandos `CREATE TABLE` na ordem correta de dependência: uma tabela só pode ser criada depois daquelas que ela referencia. A ordem da lista da Seção 2.2 já respeita isso e pode ser seguida diretamente. Não é necessário reproduzir as 17 tabelas no corpo do texto; selecionem as mais representativas – sugestão: `episodio` (entidade fraca com `UNIQUE` sobre a chave parcial), `participacao` (chave composta e `CHECK` cruzado entre colunas) e `avaliacao` com uma de suas subclasses (especialização, em que a chave primária é também chave estrangeira).*

3.3 Restrições de integridade

***Orientação:** Organizem por tipo, seguindo o item 5 do conteúdo programático, e citem a regra de negócio correspondente ao lado de cada restrição:*

- **Restrições de domínio** – `NOT NULL`, `CHECK`, `UNIQUE`. Casos a destacar: faixa e granularidade da nota (RN16); avaliação não vazia (RN17); data de exibição não futura (RN12); `funcao` e `personagem` coerentes entre si (RN10); proibição de seguir a si mesmo (RN03).

- **Integridade referencial** – justifiquem a política de cada chave estrangeira. Use-se *ON DELETE CASCADE* onde há dependência existencial (RN04, RN08, RN24) e *ON DELETE RESTRICT* onde a exclusão deve ser impedida por representar perda de informação do acervo.

3.4 Gatilhos

Orientação: Cinco gatilhos estão previstos. Para cada um, apresentem o código e – mais importante – expliquem **por que um gatilho é necessário**, isto é, por que uma restrição declarativa não resolveria. Esse é o argumento que diferencia um trabalho bom de um trabalho apenas correto.

Tabela 3.1: Gatilhos a implementar

Gatilho	Evento	Regra garantida
trg_avaliacao_exige_registro	BEFORE INSERT em avaliacao_episodio	RN18 – só se avalia episódio já registrado como assistido. Exige consulta a outra tabela, o que um CHECK não pode fazer.
trg_avaliacao_unica_por_alvo	BEFORE INSERT nas três subclasses	RN19 – uma avaliação por usuário por alvo. O autor está na superclasse e o alvo na subclasse, de modo que nenhuma restrição UNIQUE alcança as duas colunas simultaneamente.
trg_bloqueia_autocurtida	BEFORE INSERT em curtida	RN22 – ninguém curte a própria avaliação. A comparação envolve uma coluna de outra tabela.
trg_atualiza_contador_curtidas	AFTER INSERT/DELETE em curtida	Mantém o atributo derivado total_curtidas (Seção 1.7) sincronizado com a contagem real.
trg_garante_estado_assistindo	AFTER INSERT em registro	RN14 – registrar um episódio promove a série a <i>assistindo</i> . Regra de propagação de estado, inexprimível de forma declarativa.

3.5 Visões

Orientação: Duas visões estão previstas (item 4.6 do conteúdo programático). Apresentem o código e o problema que cada uma resolve.

- *vw_avaliacao_completa* – reúne a superclasse e as três subclasses em um único conjunto de linhas, resolvendo o título do alvo (nome da série, identificação da tem-

porada ou do episódio). Implementada com *UNION ALL*, exercita operações de conjunto e resolve, do lado da leitura, a fragmentação introduzida pela especialização. É a visão que as consultas da seção seguinte devem usar.

- *vw_progresso_serie* – para cada par usuário-série, apresenta episódios distintos assistidos, total de episódios e percentual de progresso. Atenção: use-se *COUNT(DISTINCT id_episodio)*, pois a RN11 permite registros repetidos do mesmo episódio e a contagem simples inflaria o progresso.

3.6 Carga de dados (DML)

Orientação: A carga precisa ser **coerente e suficiente**: coerente no sentido de respeitar todas as restrições e gatilhos (por exemplo, não tente inserir avaliação de episódio sem inserir antes o registro correspondente, ou a carga falhará por causa da RN18); suficiente no sentido de permitir que as três consultas da próxima seção retornem resultados não triviais.

Dimensionamento sugerido: 6 a 8 usuários, 5 a 6 séries com 2 a 3 temporadas cada, 6 a 10 episódios por temporada, 8 a 10 gêneros, 10 a 15 pessoas com participações variadas, um grafo de seguidores com pelo menos um caso de relação não recíproca, 40 ou mais registros de exibição, 20 ou mais avaliações distribuídas entre os três níveis, e curtidas e comentários suficientes para que a ordenação por popularidade produza um ranking com empates desfeitos.

Ordem de inserção: a mesma da criação das tabelas. Insiram os registros de exibição antes das avaliações de episódio.

3.7 Consultas de exemplo

Orientação: Três consultas que demonstrem a utilidade do banco no minimundo. Para cada uma: enunciem em português a pergunta respondida, apresentem o SQL, mostrem o resultado obtido e comentem-no. As três abaixo foram escolhidas para cobrir itens distintos do conteúdo programático – não as substituam por variações do mesmo tipo de consulta.

3.7.1 Consulta 1 – Ranking de séries por nota média

Orientação: Quais séries têm as melhores avaliações entre os usuários, considerando apenas as que receberam número significativo de notas?

Exercita junção múltipla, funções agregadas (AVG, COUNT), GROUP BY e HAVING. O HAVING COUNT() >= 3 é o ponto central: sem ele, uma série com uma única nota 5,0 lideraria o ranking, o que evidencia por que a cláusula existe. Comentem isso.*

3.7.2 Consulta 2 – Atividade recente de quem o usuário segue

Orientação: Quais foram as últimas avaliações publicadas pelos usuários que determinado usuário segue?

Exercita subconsulta aninhada (IN ou EXISTS sobre segue), auto-relacionamento e uso de visão (vw_avaliacao_completa). Vale apresentar as duas formulações – com IN e com EXISTS – e comentar a diferença.

3.7.3 Consulta 3 – Séries na lista de interesse ainda não iniciadas

Orientação: Quais séries o usuário marcou como “pretendo assistir” sem ter assistido nenhum episódio até agora?

Exercita operação de conjunto (*EXCEPT*), que é o item 4.3 do conteúdo. É a consulta que sustenta a funcionalidade real de “comece por aqui”. Vale mostrar também a formulação equivalente com *NOT EXISTS* e comentar que ambas retornam o mesmo conjunto.

3.8 Rastreabilidade das regras de negócio

Orientação: Tabela de fechamento do trabalho: para cada regra da Seção 1.2, indiquem onde ela foi implementada. Regras sem implementação precisam ser justificadas – normalmente porque pertencem à camada da aplicação. Esta tabela é a melhor defesa possível numa arguição, porque mostra que nada foi enunciado e esquecido.

Regra	Implementada por	Observações
RN01		
RN02		
RN03		
RN04		
RN05		
RN06		
RN07		
RN08		
RN09		
RN10		
RN11		
RN12		
RN13		
RN14		
RN15		
RN16		
RN17		
RN18		
RN19		
RN20		
RN21		
RN22		
RN23		
RN24		

Apêndice A

Divisão de trabalho e convenções do grupo

Este apêndice é material interno e não faz parte da entrega. Ele desaparece do PDF quando `\orientacoesfalse` é acionado.

A.1 Atribuições

Integrante	Parte	Entrega
Igor	Capítulo 1	Concluído: minimundo, 24 regras de negócio, dicionário de dados, cardinalidades, especialização, agregação e o DER.
–	Capítulo 2	Mapeamento ER → relacional, esquema relacional, dependências funcionais, demonstração de normalização e diagrama lógico.
–	Capítulo 3, seções 1–5	DDL das 17 tabelas, restrições de integridade, os cinco gatilhos e as duas visões.
–	Capítulo 3, seções 6–8	Carga de dados, as três consultas com resultados comentados e a tabela de rastreabilidade. Responsável também por executar o script inteiro do zero e confirmar que roda sem erro.

A.2 Contrato de nomes

O Capítulo 1 é a fonte de verdade para nomes de tabelas, colunas, tipos e domínios. Ninguém deve renomear nada durante a escrita dos capítulos seguintes sem atualizar o capítulo conceitual na mesma edição. O erro mais comum neste tipo de trabalho é o esquema físico divergir do diagrama conceitual – e ele é penalizado porque torna o documento internamente contraditório.

Em caso de necessidade real de mudança, anunciem ao grupo antes de alterar.

A.3 Verificação antes da entrega

1. O script SQL executa do zero, em banco vazio, sem erro e sem advertência.
2. Toda regra de negócio da Seção 1.2 aparece preenchida na tabela de rastreabilidade.
3. Os nomes no DDL coincidem com os do dicionário de dados.
4. As duas figuras de diagrama estão presentes e com os blocos descomentados.

5. Os integrantes e RAs na capa estão corretos.
6. `\orientacoestrue` foi trocado por `\orientacoesfalse`.
7. O sumário foi regenerado (compilar duas vezes).